

Raspberry Pi Zero 2 W as a headless SWL station (sdroxide)

A short, task-oriented guide to turning a **Raspberry Pi Zero 2 W** and a cheap SDR dongle into a **network listening station** for shortwave and broadcast: the radio and the decoding run on the Pi, and you drive it from a laptop, tablet or phone over your home network. Nothing here needs a licence or a transmitter.

For the listening workflow itself see the [Listening quick-start](#); for the reference detail of each control see [USER_MANUAL.md](#); for the why of this fork see the [README](#).

Nederlands: [pi-zero-2w-swl.nl.md](#). Français: [pi-zero-2w-swl.fr.md](#). Italiano: [pi-zero-2w-swl.it.md](#). PDF: [pi-zero-2w-swl.en.pdf](#).

Bold names such as **SETTINGS** and **LISTEN** are buttons and fields exactly as they appear on screen. Commands in grey boxes are typed at the Pi's shell.

Read this first. The Pi Zero 2 W is a **listening node**, not a desktop. Its four small cores and 512 MB of RAM carry the receiver, the decoders and the recording happily; they do **not** carry the wideband lanes (ADS-B, VDL2, AIS, DAB) or the graphical interface at full detail. Keep the sample rate low, keep to the narrow modes, and let the browser do the drawing.

What you need

- **A Raspberry Pi Zero 2 W** (the quad-core one — *not* the single-core Zero W, which is 32-bit and too slow). A 16 GB or larger microSD card.
 - **An SDR dongle.** An **RTL-SDR** is the easy choice: sdroxide talks to it directly, with no extra library to install. An SDRplay RSP runs too, but needs the SDRplay API service set up first — more moving parts on a small board.
 - **A powered USB hub.** This is not optional. The Zero 2 W has a single USB OTG port and a regulator that cannot reliably feed a dongle; brown-outs show up as “the radio is not found” or as dropped samples, and they are the commonest reason a Pi + SDR setup “just does not work”. Power the hub, put the dongle in the hub.
 - **A micro-USB OTG adapter** (or a hub that provides the right plug) for that single port.
 - **A wired network is best.** The Zero 2 W's Wi-Fi works, but the browser client streams the waterfall continuously and a wired link is steadier. If you use Wi-Fi, keep the waterfall at its default width.
 - **A small heatsink.** The board throttles under a sustained decode without one.
-

Part 1 — Prepare the Pi

1. Install **Raspberry Pi OS (64-bit, Bookworm)** onto the microSD card with Raspberry Pi Imager. In the Imager's settings, set the hostname (say `swl`), enable **SSH**, and enter your Wi-Fi details if you are not using a cable. Boot the Pi.
2. Log in over SSH:

```
ssh pi@swl.local
```

3. Update, and install the packages sdroxide needs:

```
sudo apt update && sudo apt full-upgrade -y  
sudo apt install -y libasound2-dev libopus-dev
```

You do **not** install a compiler. The released build is a ready-made binary.

Part 2 — Install sdroxide

The Zero 2 W is 64-bit (armv8), so it runs the **aarch64-compatible** build made for Raspberry Pi OS Bookworm. Download it on the Pi itself:

```
cd /tmp  
wget https://github.com/madmedicnl/sdroxide/releases/latest/download/sdroxide-linux-aarch64-compatible.AppImage  
chmod +x sdroxide-linux-aarch64-compatible.AppImage  
./sdroxide-linux-aarch64-compatible.AppImage --version
```

--version should print the **SDR Oxide Brown** build number. If it refuses to run with a `GLIBC_2.39 not found` message you have grabbed the wrong file — you want the one with **-compatible** in its name.

Put it where you can call it by name:

```
mkdir -p ~/bin  
mv sdroxide-linux-aarch64-compatible.AppImage ~/bin/sdroxide  
chmod +x ~/bin/sdroxide
```

If you prefer a `.deb`, the same release page has `sdroxide-linux-aarch64-compatible.deb`; install it with `sudo apt install ./sdroxide-...-aarch64-compatible.deb` and the command is then simply `sdroxide`.

The udev rules matter. A Linux user needs permission to open a USB SDR. If you installed the `.deb` they are already in place. If you used the AppImage, copy the rules from the repository's `packaging/linux/` directory to `/etc/udev/rules.d/`, then:

```
sudo udevadm control --reload && sudo udevadm trigger
```

and unplug and replug the dongle. Without this the Pi sees the dongle but cannot open it.

Part 3 — First run, on the Pi's own screen

You only need this once, to set the radio up; after that the Pi runs headless.

```
DISPLAY=:0 ~/bin/sdroxide
```

(If you have no desktop at all, skip to Part 4 and do the setup from the browser.)

1. Open **SETTINGS → Radio**. Pick your interface — for an RTL-SDR that is **RTL-SDR** — and, **importantly, set the sample rate low**: choose **1024000** or **900001**, not the 2.4 Msps default. The narrow modes need a fraction of that, and the lower rate is what keeps the Pi cool and ahead of the work. Click **Apply / reconnect**.
2. On **SETTINGS → UI**, tick **Start in SWL mode** so the transmit controls never appear — this is a listening station. (`--swl` on the command line does the same for a run.)
3. On **SETTINGS → General**, set your **Grid square** (the LISTEN window uses it for each reception's locator) and, on the **Spots** tab, your **Report as (SWL)** identity.
4. Tune a broadcast band — **LW**, **MW** or **SW** — pick **AM** or **SAM**, and confirm you hear something. The **SCHEDULE** and **LISTEN** windows are the listener's tools; the [Listening quick-start](#) walks through them.

Part 4 — Run it headless as a server

This is the point of the exercise: the Pi keeps the radio, and every screen you own can use it.

Stop the windowed instance, then start the server:

```
~/bin/sdroxide --server --port 4950
```

The browser client is served from the same port. On any device on the same network, open:

```
http://swl.local:4950
```

You get the **whole program** in the browser — tuning, the waterfall, the SCHEDULE and LISTEN windows, the decoders, the reception log. The Pi does the receiving and the decoding; the browser does the drawing, which is exactly how a 512 MB board stays useful.

A **native** sdroxide on another computer can drive it too, with `sdroxide --connect swl.local:4950`.

Keep it running

To have the server start at boot and stay up, make a small service:

```
sudo tee /etc/systemd/system/sdroxide.service >/dev/null <<'EOF'
[Unit]
Description=sdroxide SWL listening server
After=network-online.target sound.target
Wants=network-online.target

[Service]
User=pi
ExecStart=/home/pi/bin/sdroxide --server --swl --port 4950
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl enable --now sdroxide
```

Check it with `systemctl status sdroxide`, and follow its log with `journalctl -u sdroxide -f`.

What this setup does well — and what it does not

Good on a Zero 2 W:

- **Shortwave and medium-wave listening** — AM, SAM, **ECSS-U / ECSS-L**, the receive **Tone** control.
- **The listener's tools** — the **SCHEDULE** window over ~4,600 broadcasts, the **reception log** with SINPO/SIO and the QSL loop, **REPLAY**, **SIG ID**.
- **Recording on a timer** — **JOBS** records a band at a set time; each transmission becomes its own file. A node that records a programme overnight is a genuinely good use of a Zero 2 W.
- **The narrow decoders** — FT8/FT4/FT2, WSPR, JS8, PSK/RTTY, NAVTEX, DSC, ACARS.

Poor or impossible on a Zero 2 W:

- **The wideband lanes** — ADS-B, VDL2, AIS, DAB/DAB+ — need sustained megahertz of throughput and memory this board does not have. Leave them off.
- **High sample rates** (2.4 Msps and up): they cost CPU and memory for nothing the narrow modes can use. Stay at 1.024 Msps or below.
- **The graphical interface on the Pi itself**: it opens, but it is slow and competes with the DSP. Run it as a server and use a browser instead.

- **Many radio tabs at once:** each is memory and CPU. One radio is the honest load for this board.
-

If something does not work

- **“No radio found” / the dongle is missing.** Nine times in ten this is power: use the **powered hub**. The rest is the **udev rules** above.
 - **Audio crackles or the waterfall stutters.** Lower the **sample rate** on Settings → Radio, and set the waterfall width to **2048** on Settings → UI.
 - **It slows down after a few minutes.** It is throttling. Add a **heatsink** and make sure the power supply is adequate (a good 5 V 2.5 A supply, not a phone charger).
 - **The browser page will not open.** Check the server is running (`systemctl status sdioxide`) and that you are using the Pi's address (`http://swl.local:4950`, or the IP hostname -I prints).
-

Help and links

- [Listening quick-start](#) — the listener's workflow, screen by screen.
- [USER_MANUAL.md](#) — the manual, control by control.
- [README](#) — the full overview of the fork.
- [Releases](#) — the `linux-aarch64-compat` build for the Pi.

Good listening, and good DX! 73